DATA STRUCTURES AND ALGORITHMS

Case Study

Unit I - Introduction to Data Structures and Algorithms

E-commerce Product Sorting and Google Calendar Scheduling

Name	Rudrakshi Koul
Roll No.	09
College	Bharati Vidyapeeth's College of Engineering, Lavale

Case Study: E-commerce Product Sorting and Google Calendar Scheduling

Strategies: Divide and Conquer, Greedy

Problem Statement

An e-commerce platform such as Amazon or Flipkart must sort millions of products by price, rating or popularity within a fraction of a second. A calendar application such as Google Calendar must select the maximum number of non-overlapping events from a list of requests. Both problems need an algorithm that stays efficient as input grows.

Part A: Product Sorting using Merge Sort (Divide and Conquer)

- **Divide:** Split the product list into two halves.
- **Conquer:** Recursively sort each half.
- **Combine:** Merge the two sorted halves into one sorted list.
- **Complexity:** The recurrence T(n) = 2T(n/2) + n gives **O(n log n)** time in best, average and worst cases, with O(n) extra space.
- **Stability:** Merge sort is stable, so products with equal prices keep their earlier order (for example, sorted by rating).

Part B: Calendar Scheduling using a Greedy Strategy (Activity Selection)

- Sort all events by **finish time**.
- Select the first event, then repeatedly pick the next event whose start time is at or after the finish time of the last selected event.
- **Complexity:** O(n log n) for sorting plus O(n) for selection.
- **Why it works:** Choosing the earliest-finishing event leaves the most room for later events.

Event	Start	End	Selected?
Lecture	9:00	10:00	Yes
Lab	9:30	11:00	No (overlaps)
Meeting	10:00	11:00	Yes
Workshop	10:30	12:00	No (overlaps)
Seminar	11:00	12:00	Yes

Conclusion: Divide and Conquer gives efficient sorting for large datasets, while the Greedy strategy gives optimal scheduling through simple local decisions.

Python Program

```
# Unit I: Merge Sort (Divide and Conquer) + Activity Selection (Greedy)
def merge_sort(items, key):
    if len(items) <= 1:
        return items
    mid = len(items) // 2
```

```
    left = merge_sort(items[:mid], key)
    right = merge_sort(items[mid:], key)
    merged, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if key(left[i]) <= key(right[j]):      # <= keeps the sort stable
            merged.append(left[i]); i += 1
        else:
            merged.append(right[j]); j += 1
    return merged + left[i:] + right[j:]

def select_events(events):
    events = sorted(events, key=lambda e: e[2])   # sort by finish time
    chosen, last_end = [], 0
    for name, start, end in events:
        if start >= last_end:
            chosen.append(name); last_end = end
    return chosen

products = [("Phone", 15999), ("Earbuds", 1299), ("Laptop", 54999),
            ("Charger", 799), ("Watch", 2499)]
print("Products sorted by price:")
for name, price in merge_sort(products, key=lambda p: p[1]):
    print(f"  {name:8} Rs.{price}")

events = [("Lecture", 9, 10), ("Lab", 9.5, 11), ("Meeting", 10, 11),
          ("Workshop", 10.5, 12), ("Seminar", 11, 12)]
print("Events selected:", select_events(events))
```

Output

```
Products sorted by price:
  Charger  Rs.799
  Earbuds  Rs.1299
  Watch    Rs.2499
  Phone    Rs.15999
  Laptop   Rs.54999
Events selected: ['Lecture', 'Meeting', 'Seminar']
```